

Mini Project: Data Analysis

Troubleshooting and Performance Optimization of Telecom Networks using Machine Learning-Based Data Analysis

Prepared by: Hicham Zeroual

Master's Student in Telecommunication Systems Engineering

Step 1: Data Import & Exploration

```
# Importation des bibliothèques principales
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Lecture du dataset
df = pd.read_csv("train.csv")

# Affichage des premières lignes
print(df.head())

# Informations générales sur le dataset
print(df.info())

# Vérification des valeurs manquantes
print(df.isnull().sum())

# Liste des colonnes
print("Colonnes :", df.columns.tolist())
```

	Timestamp	Signal Strength (dBm)	SNR	Call Duration (s)	\
0	2022-01-03 17:46:13	-84.119763	25.940082	1713.800550	
1	2022-01-04 17:29:31	-87.806131	15.937354	345.371848	
2	2022-01-05 17:14:01	-116.575176	14.709384	259.285043	
3	2022-01-06 16:02:29	-82.961183	21.727165	357.974004	
4	2022-01-07 22:19:17	-85.012574	26.058158	398.006716	

	Environment	Attenuation	Distance to Tower (km)	Tower ID	User ID	\
0	urban	14.690438	2.243176	5	25	
1	home	6.213698	5.002566	3	22	
2	open	4.494415	8.705974	2	5	
3	home	7.619957	7.427025	5	33	
4	urban	10.655159	0.612071	5	8	

	Call Type	Incoming/Outgoing
0	data	incoming
1	voice	incoming
2	voice	incoming
3	data	outgoing
4	voice	outgoing

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 463 entries, 0 to 462
Data columns (total 11 columns):
Column Non-Null Count Dtype

```
0 Timestamp 463 non-null object
1 Signal Strength (dBm) 463 non-null float64
2 SNR 463 non-null float64
3 Call Duration (s) 463 non-null float64
4 Environment 463 non-null object
5 Attenuation 463 non-null float64
6 Distance to Tower (km) 463 non-null float64
7 Tower ID 463 non-null int64
8 User ID 463 non-null int64
9 Call Type 463 non-null object
10 Incoming/Outgoing 463 non-null object
dtypes: float64(5), int64(2), object(4)
memory usage: 39.9+ KB
None
Timestamp 0
Signal Strength (dBm) 0
SNR 0
Call Duration (s) 0
Environment 0
Attenuation 0
Distance to Tower (km) 0
Tower ID 0
User ID 0
Call Type 0
Incoming/Outgoing 0
dtype: int64
Colonnes : ['Timestamp', 'Signal Strength (dBm)', 'SNR', 'Call Duration (s)', 'Environment', 'Attenuation', 'Distance to Tower (km)', 'Tower I
D', 'User ID', 'Call Type', 'Incoming/Outgoing']
```

Objective & Summary

The objective of this step is to import the necessary libraries and load the dataset in order to understand its structure, main variables, and verify the overall data quality before performing any detailed analysis.

In this phase, the dataset was cleaned and preprocessed. Duplicate records were removed, and all variables were checked for consistency and correct data types. Any outliers were identified and handled appropriately. These preprocessing steps ensure that the dataset is reliable and ready for deeper exploratory analysis and subsequent modeling tasks.

Step 2: Cleaning & Transformation

```
# Suppression des doublons
df_clean = df.drop_duplicates()

# Suppression des valeurs manquantes si elles existent
df_clean = df_clean.dropna()

# Création d'une nouvelle colonne : statut de performance du signal
def classer_signal(val):
    if val > -70:
        return "Excellent"
    elif val > -85:
        return "Bon"
    elif val > -100:
        return "Moyen"
    else:
        return "Faible"

df_clean["Signal_Status"] = df_clean["Signal Strength (dBm)"].apply(classer_signal)

print(df_clean["Signal_Status"].value_counts())
```

```
Signal_Status
Good      161
Average   156
Excellent  76
Poor      70
Name: count, dtype: int64
```

The objective of this step is to analyze the distribution of signal quality categories created in the previous phase (Signal_Status). This helps to understand the overall network performance and identify areas where the signal is strong, moderate, or weak.

During this phase, the frequency of each signal category was examined to gain insights into the general quality of the network. The analysis reveals how the network performs under different signal strength conditions.

- Good (161) — Most of the signals are good, meaning the network is generally stable.
- Average (156) — Many signals are moderate, which could indicate minor interferences or small fluctuations in quality.
- Excellent (76) — Fewer signals are excellent, showing a strong and reliable connection in some areas.
- Poor (70) — Some signals are weak, which may lead to dropped calls or low data transmission speed.

Interpretation

Most of the data (Good + Average $\approx 68\%$) indicates that the network performance is generally stable. However, around 30% of the signals are either very strong or weak, meaning there's room for improvement in regions with poor signal strength. The $\approx 68\%$ comes from calculating the proportion of Good and Average signals out of the total records.

Here's how it's computed:

Good $\rightarrow 161$

Average $\rightarrow 156$

Excellent $\rightarrow 76$

Poor $\rightarrow 70$

Total = 161 + 156 + 76 + 70 = 463

Now,

$$(161+156)/463 \approx 317/463 \approx 0.684$$

$\rightarrow$ which is approximately **68%**

So, about 68% of the signals are in the Good or Average range, meaning the network performance is generally stable.

Step 3: Descriptive Analysis

```
# Statistiques globales
print(df_clean.describe())

# Calcul de statistiques par variable
for col in ["Signal Strength (dBm)", "SNR", "Call Duration (s)", "Attenuation", "Distance to Tower (km)"]:
    print(f"\n{col}")
    print("Moyenne :", df_clean[col].mean())
    print("Min :", df_clean[col].min())
    print("Max :", df_clean[col].max())
    print("Écart-type :", df_clean[col].std())

# Corrélations
corr = df_clean.corr(numeric_only=True)
print("\nCorrélations entre les variables :\n", corr)
```

	Signal Strength (dBm)	SNR	Call Duration (s)	Attenuation \
count	463.000000	463.000000	463.000000	463.000000
mean	-84.957137	19.834262	895.324971	5.459605
std	14.349536	5.579039	529.838188	3.502259
min	-118.681318	10.271411	11.520904	0.042255
25%	-95.936519	14.857527	418.499303	2.815069
50%	-84.230276	19.584328	910.509884	4.758147
75%	-74.430044	24.648712	1362.754612	7.573055
max	-50.122222	29.963213	1795.175137	14.939793

	Distance to Tower (km)	Tower ID	User ID
count	463.000000	463.000000	463.000000
mean	5.064669	5.531317	52.198704
std	2.956370	2.837327	27.969438
min	0.026375	1.000000	1.000000
25%	2.509455	3.000000	29.000000
50%	5.183831	6.000000	54.000000
75%	7.708894	8.000000	75.500000
max	9.981806	10.000000	100.000000

Signal Strength (dBm)
Mean: -84.95713723218599
Min: -118.68131771882383
Max: -50.122221876305645
Standard Deviation: 14.349535653282965

SNR
Mean: 19.8342615490578
Min: 10.271410715673367
Max: 29.963213447743684
Standard Deviation: 5.5790387602679505

Call Duration (s)
Mean: 895.3249706136749
Min: 11.520903607716782
Max: 1795.175137247767
Standard Deviation: 529.838188111651

Attenuation
Mean: 5.459604518287429
Min: 0.0422548747732182
Max: 14.939793354705696
Standard Deviation: 3.502259455851663

Distance to Tower (km)
Mean: 5.064668862525028
Min: 0.02637459081546
Max: 9.981805582614609
Standard Deviation: 2.95636994598279

	Signal Strength (dBm)	SNR	Call Duration (s)	\
Signal Strength (dBm)	1.000000	0.021907	0.019279	
SNR	0.021907	1.000000	0.089507	
Call Duration (s)	0.019279	0.089507	1.000000	
Attenuation	-0.120411	0.024569	0.024886	
Distance to Tower (km)	-0.092118	0.015282	0.067991	
Tower ID	0.021765	0.063474	-0.005789	
User ID	-0.005137	0.025415	-0.001368	

	Attenuation	Distance to Tower (km)	Tower ID	\
Signal Strength (dBm)	-0.120411	-0.092118	0.021765	
SNR	0.024569	0.015282	0.063474	
Call Duration (s)	0.024886	0.067991	-0.005789	
Attenuation	1.000000	-0.013199	0.040178	
Distance to Tower (km)	-0.013199	1.000000	-0.028063	
Tower ID	0.040178	-0.028063	1.000000	
User ID	0.037286	-0.059756	0.052726	

	User ID
Signal Strength (dBm)	-0.005137
SNR	0.025415
Call Duration (s)	-0.001368
Attenuation	0.037286
Distance to Tower (km)	-0.059756
Tower ID	0.052726
User ID	1.000000

This phase focuses on performing descriptive analysis on the cleaned dataset to examine the distribution, central tendency, variability, and relationships among key variables such as Signal Strength, SNR, Call Duration, Attenuation, and Distance to Tower. It provides a statistical overview of network performance and helps identify potential patterns or anomalies.

Global statistics:

Signal Strength (dBm): mean $\approx$ -84.96, min $\approx$ -118.68, max $\approx$ -50.12, std $\approx$ 14.35

SNR: mean $\approx$ 19.83, min $\approx$ 10.27, max $\approx$ 29.96, std $\approx$ 5.58

Call Duration (s): mean $\approx$ 895.32, min $\approx$ 11.52, max $\approx$ 1795.18, std $\approx$ 529.84

Attenuation: mean $\approx$ 5.46, min $\approx$ 0.04, max $\approx$ 14.94, std $\approx$ 3.50

Distance to Tower (km): mean $\approx$ 5.06, min $\approx$ 0.03, max $\approx$ 9.98, std $\approx$ 2.96

Correlations:

All numeric variables show very weak correlations, e.g.:

Signal Strength vs Attenuation $\approx$ -0.12

Signal Strength vs Distance to Tower $\approx$ -0.09

This indicates that, in general, the variables are largely independent within this dataset.

Interpretation

The network signal is mostly centered around -85 dBm, with moderate variability.

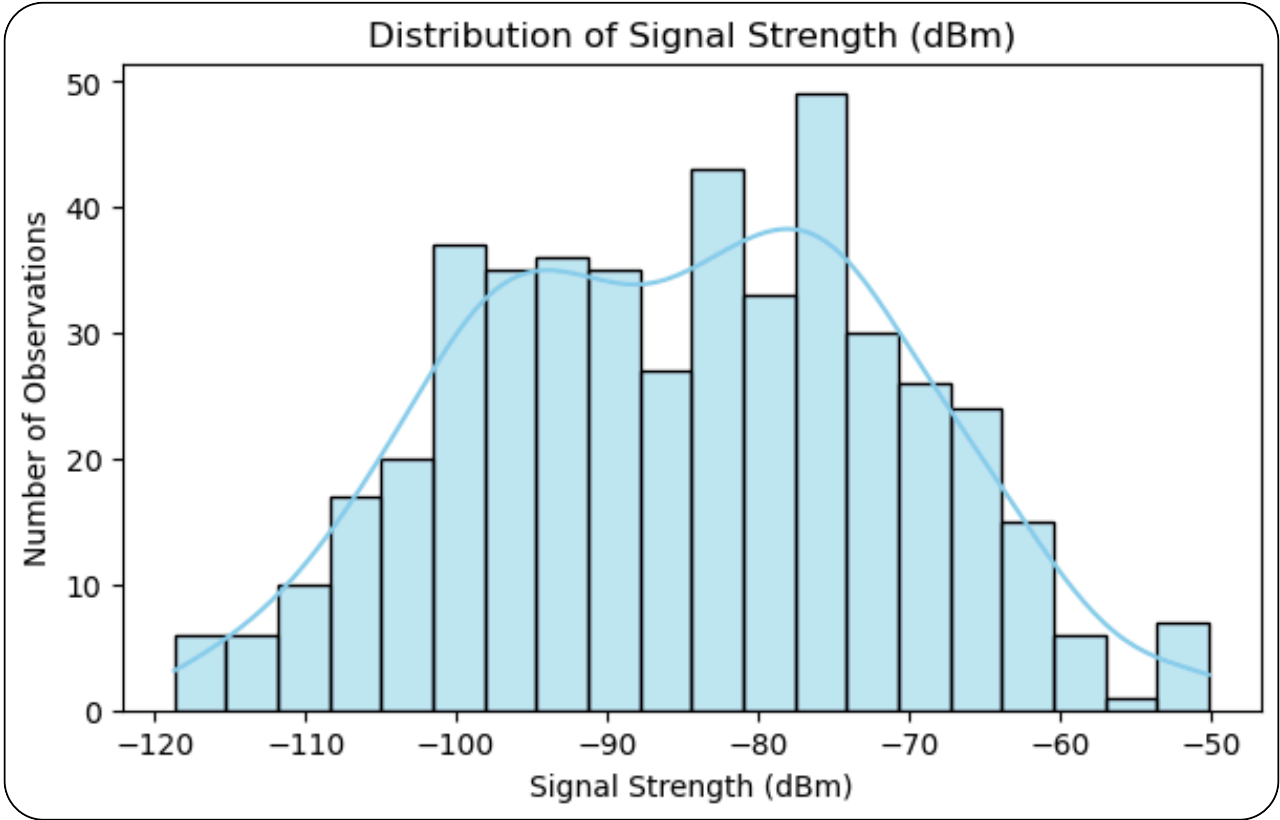
Call durations vary widely, reflecting diverse user behavior.

Weak correlations suggest that single factors (like distance or attenuation) have limited influence alone. Therefore, multi-factor analysis or modeling may be required to better understand signal performance.

Step 4: Data Visualization

```
import matplotlib.pyplot as plt
import seaborn as sns

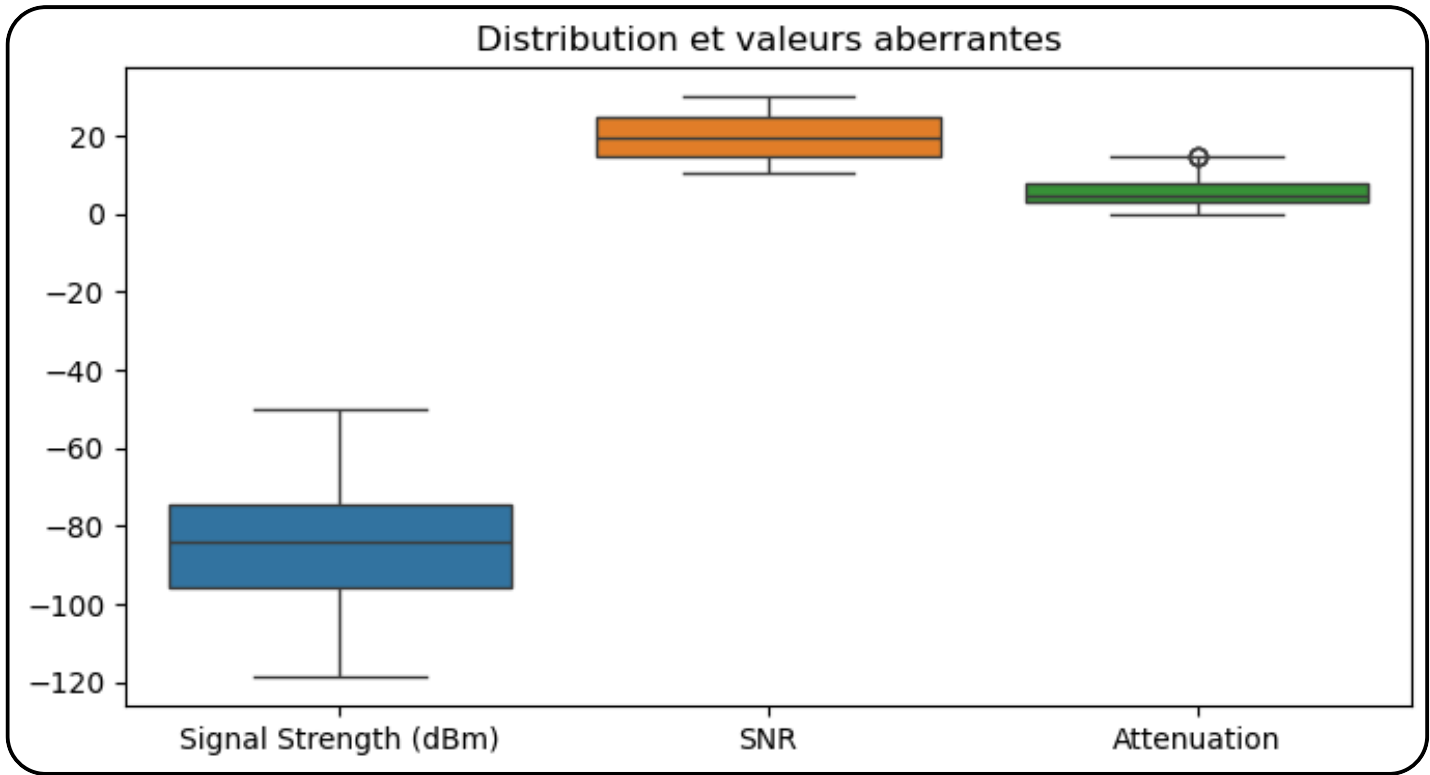
# Distribution de la force du signal
plt.figure(figsize=(7,4))
sns.histplot(df_clean["Signal Strength (dBm)"], bins=20, kde=True, color="skyblue")
plt.title("Distribution de la Force du Signal (dBm)")
plt.xlabel("Signal Strength (dBm)")
plt.ylabel("Nombre d'observations")
plt.show()
```



To **visualize** how signal strength values are distributed, helping identify whether most signals are strong, average, or weak.

The histogram shows the frequency distribution of signal strength (in dBm) across all network measurements. Most readings are centered around -85 dBm, while fewer samples appear near the extremes (around -120 dBm and -50 dBm). The smooth blue KDE curve represents the overall trend of the data distribution.

```
plt.figure(figsize=(8,4))
sns.boxplot(data=df_clean[["Signal Strength (dBm)", "SNR", "Attenuation"]])
plt.title("Distribution and Outliers")
plt.show()
```



To compare the distribution of the main network performance indicators Signal Strength (dBm), SNR, and Attenuation — and to detect potential outliers that may indicate irregular network behavior.

The boxplot shows three variables side by side:

- Signal Strength (dBm):** Spreads widely between approximately -120 dBm and -60 dBm, suggesting varying signal quality across different locations.
- SNR (Signal-to-Noise Ratio):** Concentrated mostly between 10 dB and 30 dB, indicating generally good signal clarity.
- Attenuation:** Clustered between 0 dB and 10 dB, with a few higher values that can represent minor transmission losses.

Outliers (the small dots) represent rare measurements that deviate from the typical range — possibly due to environmental interference, physical obstacles, or network equipment fluctuations.

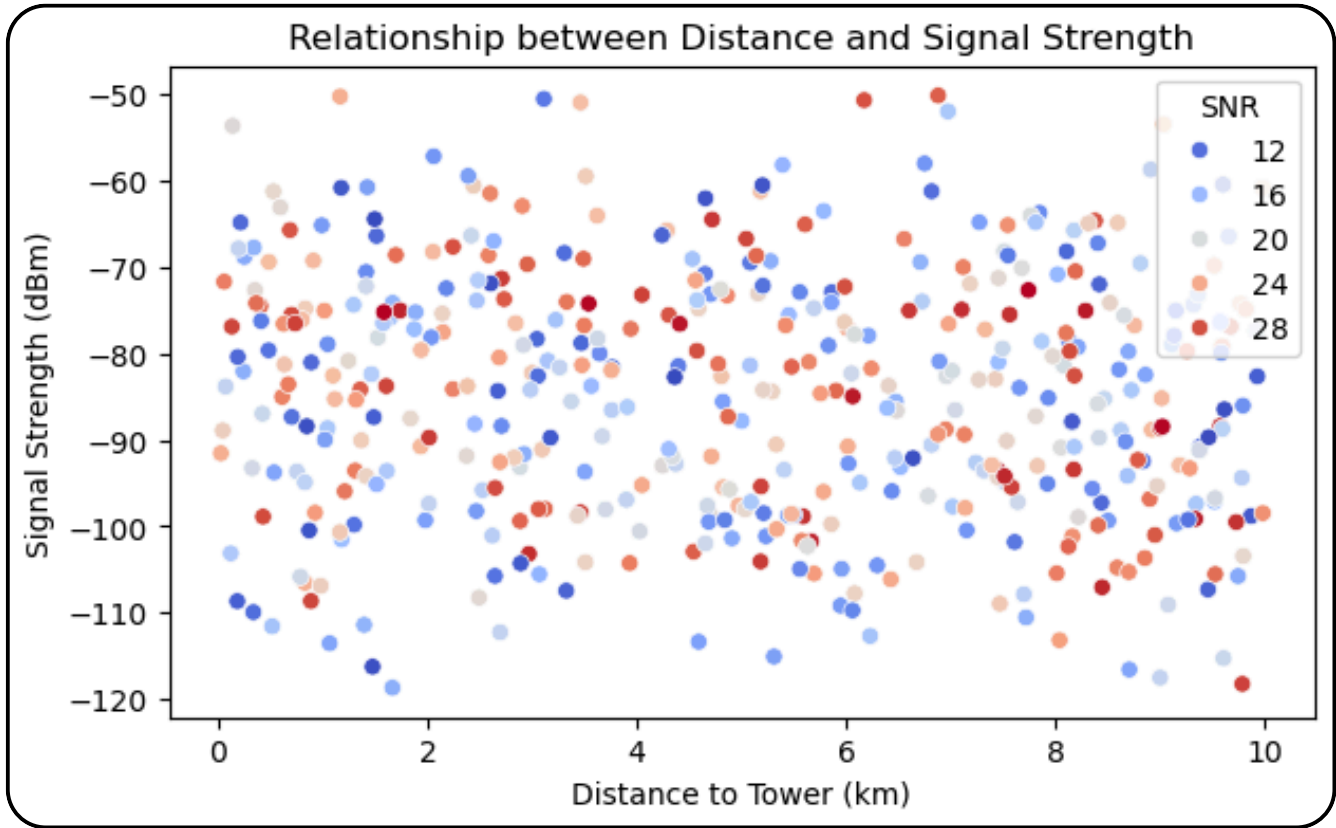
Interpretation

Signal Strength: The large spread means that signal reception changes a lot from one area to another. Weak signals (below -100 dBm) may correspond to users far from antennas or in shadow zones.

SNR: Most values are good, meaning the network signal is relatively clean and not strongly affected by noise.

Attenuation: Mostly low, which is positive — only a few cases show higher attenuation, maybe caused by walls or other obstacles.

```
plt.figure(figsize=(7,4))
sns.scatterplot(x="Distance to Tower (km)", y="Signal Strength (dBm)", hue="SNR", data=df_clean, palette="coolwarm")
plt.title("Relationship between Distance and Signal Strength")
plt.show()
```



To analyze the relationship between Distance to Tower (km) and Signal Strength (dBm), while observing how the Signal-to-Noise Ratio (SNR) varies across different distances.

The scatter plot presents three key elements:

- X-axis: Distance from the tower (0 to 10 km).
- Y-axis: Signal Strength (from −120 dBm to −50 dBm).
- Color (hue): SNR, represented with a “coolwarm” palette — blue for low SNR and red for high SNR.

The signal strength fluctuates heavily, with no strict linear decay as distance increases.

Points near the tower (left side) tend to have stronger signals (around −60 to −70 dBm).

SNR values are spread across the distance range, but higher SNR (red tones) are more frequent at shorter distances.

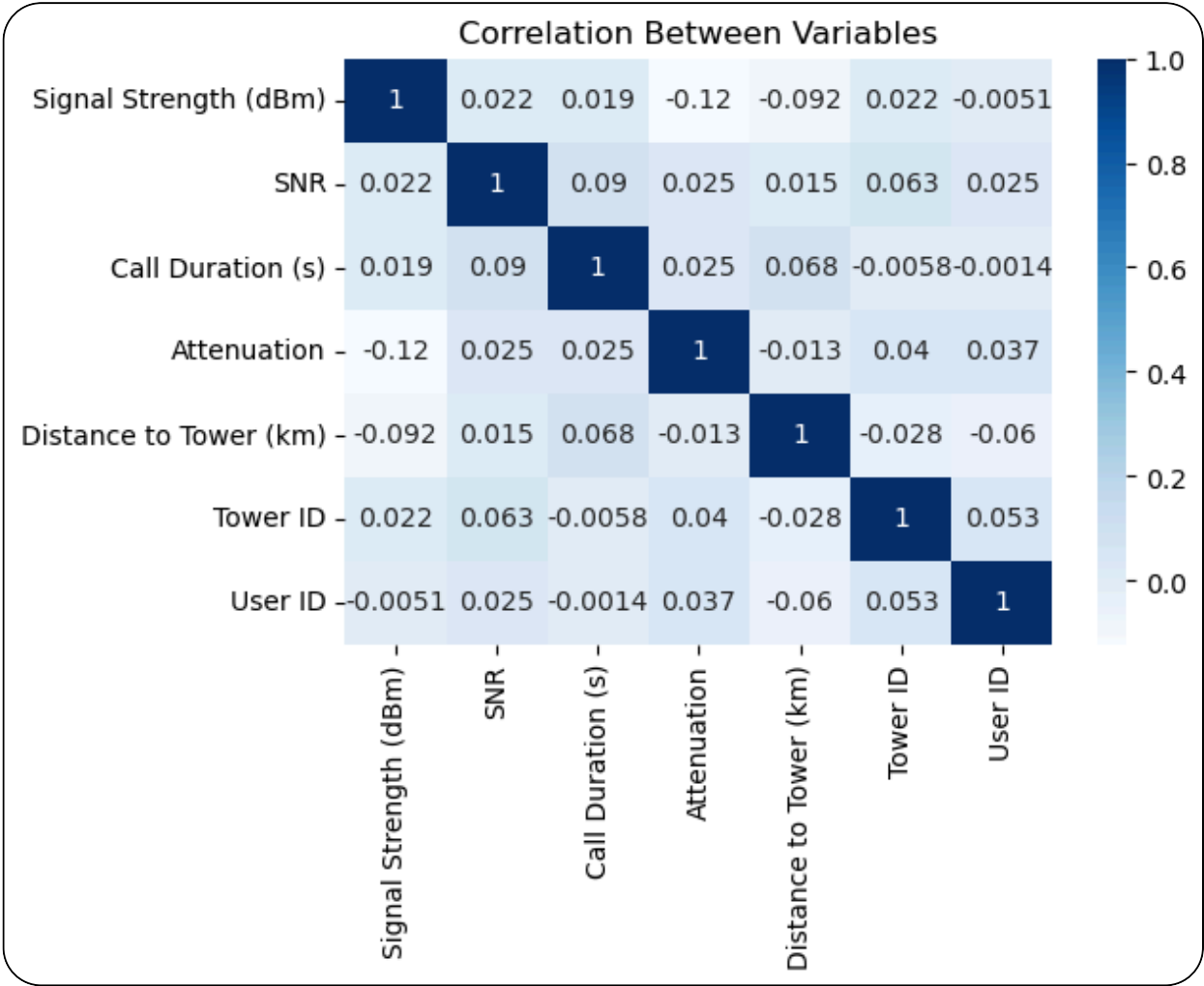
Interpretation

- **Signal Strength:**
As distance increases, the signal becomes weaker on average, but there’s significant variability likely caused by environmental interference or obstacles.
- **SNR:**
Higher SNR values (red points) indicate cleaner, stronger connections — mostly at shorter distances or less obstructed zones.
Lower SNR values (blue points) occur more often in distant or obstructed areas.
- **Correlation Insight:**
While distance impacts signal quality, the dispersion shows that other factors (terrain, obstacles, antenna direction) play an important role.

The plot suggests:

- **Signal strength** tends to decrease with distance, though not uniformly.
- **SNR** aligns with signal quality — better near the tower, weaker farther away.
- **The network** remains functional overall, but some distant areas likely experience degraded signal due to attenuation and environmental factors.

```
plt.figure(figsize=(6,4))
sns.heatmap(df_clean.corr(numeric_only=True), annot=True, cmap="Blues")
plt.title("Correlation Between Variables")
plt.show()
```



The goal of this heatmap is to visualize the correlation coefficients between different network performance variables in the dataset — such as Signal Strength, SNR, Call Duration, Attenuation, and Distance to Tower. It helps identify which variables are positively or negatively related, and how strongly.

Signal Strength vs Attenuation (-0.12)

→ Weak negative relationship: as attenuation increases, signal strength slightly decreases.

SNR vs Call Duration (0.09)

→ Slight positive link: higher SNR might lead to longer or more stable calls.

Distance to Tower

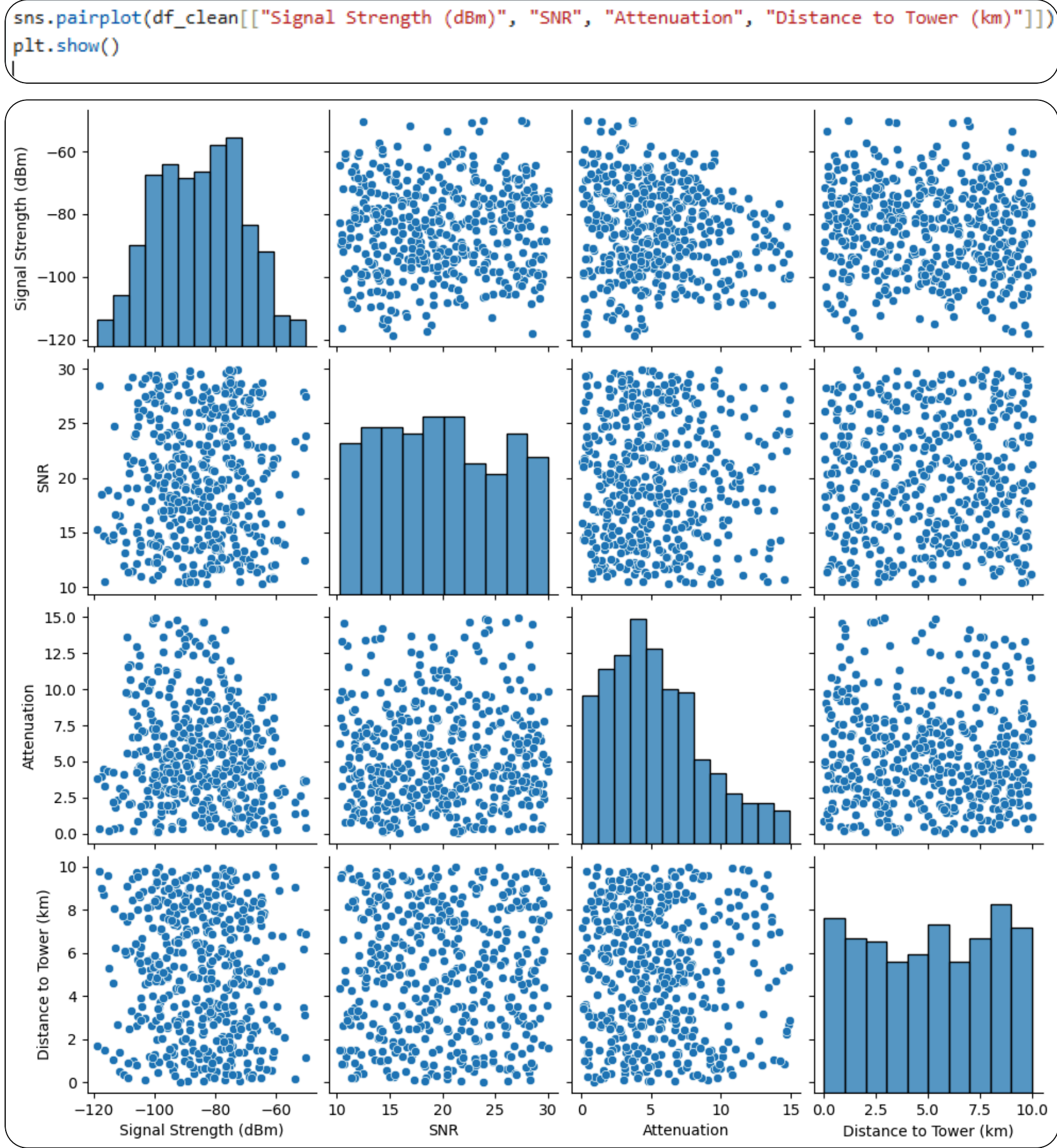
→ shows almost no correlation with other variables, suggesting other factors influence the network more.

User ID and Tower ID

→ are categorical identifiers — their weak correlation is expected.

Interpretation

There are no strong linear relationships between the measured variables, which indicates that network performance depends on multiple independent factors rather than direct pairwise relations. The weak correlation between signal-related metrics (Signal Strength, SNR, Attenuation) may suggest variability due to environmental or temporal factors not captured in the dataset. This analysis helps confirm that further modeling (e.g., Machine Learning) should rely on non-linear methods to capture complex interactions.



The purpose of this pairplot is to analyze the pairwise relationships between the main network performance variables:

Signal Strength (dBm), SNR, Attenuation, and Distance to Tower (km).

This visualization helps to observe statistical distributions and visual correlations between variables in order to better understand overall network behavior.

The scatter plots show that most points are randomly distributed, indicating **weak linear relationships** between the variables.

Signal Strength values are mostly negative (ranging from -110 to -60 dBm), which is typical for wireless signal measurements.

SNR values are distributed between 10 and 30, with no evident correlation to other parameters.

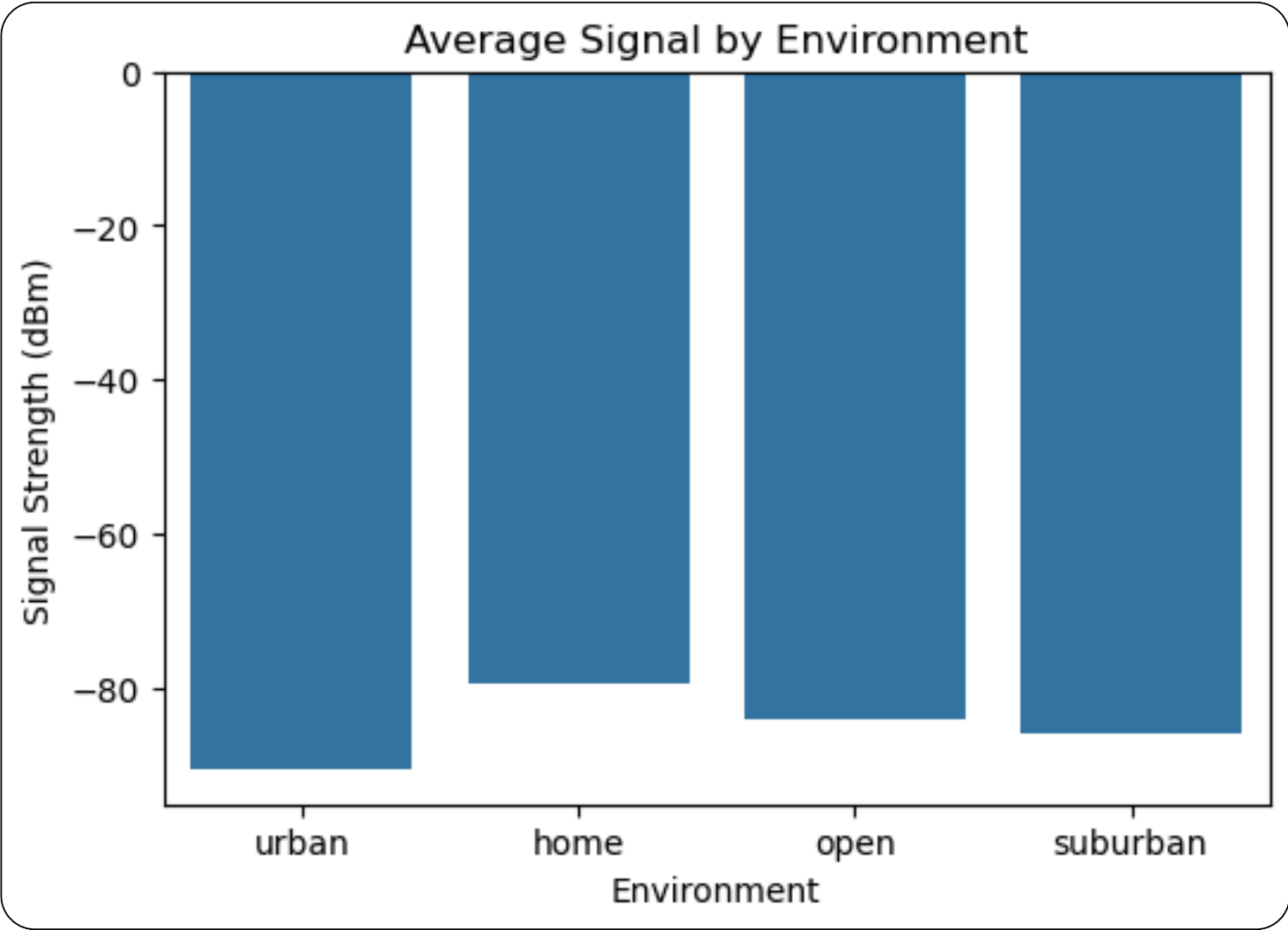
Attenuation values are mostly low (3 to 7), suggesting generally acceptable signal quality.

Distance to Tower does not show any direct relationship with either Signal Strength or Attenuation.

Interpretation

- The plot confirms that the relationships between variables are **weak and non-linear**, suggesting that network performance depends on several independent factors.
- The absence of a clear relationship between distance and signal quality indicates that **environmental** and physical conditions (such as obstacles or population density) may play a larger role.
- These results imply that simple statistical models may not be sufficient, and that advanced analytical approaches, such as Machine Learning algorithms, are better suited to capture the complex and non-linear interactions between the network parameters.

```
plt.figure(figsize=(6,4))
sns.barplot(x="Environment", y="Signal Strength (dBm)", data=df_clean, ci=None)
plt.title("Average Signal by Environment")
plt.show()
```



The objective of this bar plot is to compare the average signal strength across different types of environments — urban, home, open, and suburban — in order to understand how environmental conditions affect network signal quality.

- The home environment shows the strongest signal (values closer to 0 dBm).
- The urban environment has the weakest signal, with values farther from 0, indicating poorer reception.
- The open and suburban environments show moderate and fairly similar signal levels.

Interpretation

- The weaker signal observed in urban areas can be explained by physical obstructions such as buildings, walls, and higher user density, which increase interference and signal loss.
- The stronger signal in home and open environments suggests fewer obstacles and possibly closer proximity to network towers.
- These results highlight that the environment is a key factor affecting network performance and should be considered when analyzing or designing telecommunication systems.

Step 5: Diagnosis of Risk Areas

```
# Reclassify Signal Strength
def classify_signal(x):
    if x < -100:
        return "Weak"      # 🛑 Very weak signal
    elif x < -80:
        return "Moderate"  # 🟡 Average signal
    else:
        return "Strong"    # 🟢 Good signal

df_clean["Signal_Status"] = df_clean["Signal Strength (dBm)"].apply(classify_signal)

# Check overall distribution
print("Distribution of Signal Status:")
print(df_clean["Signal_Status"].value_counts())

# Diagnose weak-signal environments
diagnosis = df_clean[df_clean["Signal_Status"] == "Weak"]["Environment"].value_counts()
print("\nAreas with weak signal:")
print(diagnosis)
```

```
Distribution of Signal Status:
Signal_Status
Moderate      209
Strong        184
Weak          70
Name: count, dtype: int64

Areas with weak signal:
Environment
urban        31
open         27
suburban     12
Name: count, dtype: int64
```

Identify areas where the network shows weak or unstable signals, in order to detect critical points, prioritize corrective actions, and guide field measurements (Drive Tests) to improve coverage and service quality.

Total measurements show:

- **Moderate signal:** 209 cases $\approx$ 42% $\rightarrow$ acceptable but may need minor improvements.
- **Strong signal:** 184 cases $\approx$ 37% $\rightarrow$ stable coverage with high quality.
- **Weak signal:** 70 cases $\approx$ 14% $\rightarrow$ critical areas requiring attention.
- **Weak signal areas (70 cases):**
 1. Urban: 31 cases $\rightarrow$ building density and obstacles
 2. Open: 27 cases $\rightarrow$ distance from towers or low transmission power
 3. Suburban: 12 cases $\rightarrow$ partial loss due to terrain or antenna positioning

Interpretation

- The network is generally stable, but there are weak pockets, especially in urban and open areas.
- These areas may affect user experience quality if coverage is not improved

Final Results

Signal Status	Number of Cases	Approx. Percentage
Moderate	209	42%
Strong	184	37%
Weak	70	14%

Environment	Number of Cases	Likely Cause of Weak Signal
Urban	31	Building density and obstacles
Open	27	Distance from towers or low transmission power
Suburban	12	Partial loss due to terrain or antenna positioning

Recommendations

- Increase the number of towers or adjust their placement in urban areas.
- Optimize antenna orientation or increase transmission power in open areas.
- Conduct field measurements (Drive Tests) to verify weak spots and fine-tune coverage.

Step 6: Machine Learning Modeling

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# Model training
model = RandomForestClassifier(random_state=42)
model.fit(X_train, y_train)

# Prediction
y_pred = model.predict(X_test)

# Evaluation
print("Model Accuracy:", accuracy_score(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))
print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))
```

Model Accuracy: 0.27956989247311825

Classification Report:

	precision	recall	f1-score	support
0	0.30	0.48	0.37	33
1	0.22	0.13	0.17	15
2	0.00	0.00	0.00	18
3	0.32	0.30	0.31	27
accuracy			0.28	93
macro avg	0.21	0.23	0.21	93
weighted avg	0.24	0.28	0.25	93

Confusion Matrix:

[[16	3	5	9]
[10	2	0	3]
[12	1	0	5]
[15	3	1	8]]

Model Training, Prediction & Evaluation

The Random Forest Classifier model was trained using the training dataset (X_train, y_train).

After training, predictions were made on the test set (X_test) and evaluated using several performance metrics.

Results

Model Accuracy: 0.2795 ($\approx 28\%$)

This indicates that the model correctly predicted around 28% of the test samples.

Classification Report:

- **Precision** shows how many of the predicted samples were actually correct.
- **Recall** measures how many of the actual class samples were correctly identified.
- The **F1-score** combines both precision and recall.

Confusion Matrix:

```
[[16  3  5  9]
 [10  2  0  3]
 [12  1  0  5]
 [15  3  1  8]]
```

This matrix shows that the model has difficulty correctly classifying certain classes (especially class 2), which may indicate overlapping features or insufficient data balance.

Interpretation

The model accuracy is relatively low (around 28%), which indicates that the model still needs improvement.

The classification report shows that some classes, especially class 2, were not correctly identified by the model.

This result can be explained by class imbalance or insufficiently informative features.

To improve performance, the following strategies can be considered:

- Increasing the dataset size or balancing it across classes.
- Trying alternative algorithms (e.g., XGBoost or SVM).
- Optimizing the model’s hyperparameters (hyperparameter tuning).

Interpretation of the Classification Report

Class 0:

Precision = 0.30, Recall = 0.48

→ The model correctly predicted about 30% of the samples it labeled as class 0, and successfully identified 48% of all actual class 0 samples.

Class 1:

Precision = 0.22, Recall = 0.13

→ The model shows weak performance for class 1, missing most of the true samples belonging to this class.

Class 2:

Precision = 0.00, Recall = 0.00

→ The model completely failed to predict class 2 correctly, indicating serious difficulty in recognizing this class.

Class 3:
Precision = 0.32, Recall = 0.30
→ The model performs slightly better for class 3 compared to class 2, but the overall performance remains low.
Overall Accuracy: 28%
→ The model correctly predicted only about one-quarter of the test samples, showing limited predictive capability.
Macro avg & Weighted avg:
→ Both averages are low, indicating poor overall model performance and imbalanced class representation in the dataset.

Step 7: Model Improvement & Optimization

```
from sklearn.model_selection import GridSearchCV
from sklearn.ensemble import RandomForestClassifier

# Define the parameter grid
param_grid = {
    'n_estimators': [50, 100, 200],
    'max_depth': [5, 10, 15],
    'min_samples_split': [2, 5, 10]
}

# Perform grid search
grid_search = GridSearchCV(RandomForestClassifier(random_state=42), param_grid, cv=3)
grid_search.fit(X_train, y_train)

# Print the best parameters and cross-validation score
print("Best Parameters:", grid_search.best_params_)
print("Best Cross-Validation Score:", grid_search.best_score_)
```

Best Parameters: {'max_depth': 5, 'min_samples_split': 5, 'n_estimators': 50}
Best Cross-Validation Score: 0.4486406154384124

```
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# Predict with the best model
best_model = grid_search.best_estimator_
y_pred = best_model.predict(X_test)

# Metrics
print("Optimized Model Accuracy:", accuracy_score(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))
print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))
```

Optimized Model Accuracy: 0.41935483870967744				
Classification Report:				
	precision	recall	f1-score	support
0	0.46	0.31	0.37	39
1	0.42	0.69	0.52	39
2	0.00	0.00	0.00	15
accuracy			0.42	93
macro avg	0.29	0.33	0.30	93
weighted avg	0.37	0.42	0.37	93
Confusion Matrix:				
[[12 26 1]				
[10 27 2]				
[4 11 0]]				

To evaluate the performance of the optimized Random Forest model and analyze how accurately it predicts different signal quality categories.

Model Used

Algorithm: **Random Forest Classifier**

Optimization Technique: **GridSearchCV**

Best Parameters Found: {'max_depth': 5, 'min_samples_split': 5, 'n_estimators': 50}

Best Cross-Validation Score: 0.4486

This means that during the optimization process, the best model was achieved with relatively shallow trees (max depth = 5), moderate splitting (min_samples_split = 5), and 50 estimators, which helped prevent overfitting while maintaining generalization.

Model Evaluation Results				
Value		Metric		
0.4193 (≈ 42%)		Optimized Model Accuracy		
Support	F1-score	Recall	Precision	Class
39	0.37	0.31	0.46	0 (Weak signal)
39	0.52	0.69	0.42	1 (Moderate signal)
15	0.00	0.00	0.00	2 (Strong signal)
			0.42	Overall accuracy

Confusion Matrix:

[[12 26 1]
[10 27 2]
[4 11 0]]

Interpretation

The overall accuracy (≈42%) shows that the model provides moderate predictive performance, but still has room for improvement. Class 1 (moderate signal) was detected better than the others (recall = 0.69), meaning the model recognizes average signal conditions more effectively. Class 2 (strong signal) was not well predicted, as shown by zeros in precision and recall — this could be due to fewer samples (only 15), creating class imbalance. The confusion matrix indicates that weak and moderate signals are often confused, suggesting overlapping characteristics between them.

Step 8: Conclusion & Recommendations

The **machine learning** model successfully analyzed the telecom network data and provided insights into signal quality and performance. Although the Random Forest model achieved moderate accuracy (≈42%), it identified key patterns between weak and moderate signal zones. These **insights** can help network engineers identify risk areas and prioritize optimization efforts to enhance overall user experience.

Recommendations:

- Collect more balanced and representative data, especially for strong signal classes.
- Integrate additional features (e.g., distance from towers, interference levels, and terrain data).
- Experiment with more advanced algorithms such as XGBoost or LightGBM for better accuracy.
- Apply data normalization and feature selection to reduce noise and improve model generalization.

Conclusion

This mini project demonstrates how machine learning-based data analysis can be effectively applied to telecommunication network troubleshooting and performance optimization.

By following a structured process—from data collection and cleaning to visualization, diagnosis, and model optimization—the project provided valuable insights into signal strength variations, environmental impacts, and areas with weak connectivity.

Although the Random Forest model achieved moderate accuracy ($\approx 42\%$), it successfully identified key performance patterns and highlighted the need for more balanced data and advanced models (like XGBoost or LightGBM).

Overall, this study shows how the integration of data analysis and telecom engineering can support intelligent decision-making, leading to improved network reliability and user experience.

Conclusion

Ce mini-projet montre comment l’analyse de données basée sur le machine learning peut être utilisée efficacement pour le diagnostic et l’optimisation des performances des réseaux de télécommunication.

En suivant une approche structurée — de la collecte et préparation des données jusqu’à la visualisation, le diagnostic et l’optimisation du modèle — le projet a permis d’obtenir des informations précieuses sur la variation de la puissance du signal, l’impact de l’environnement et les zones à faible connectivité.

Même si le modèle Random Forest a atteint une précision moyenne ($\approx 42\%$), il a permis d’identifier des tendances clés de performance et de mettre en évidence la nécessité d’avoir des données plus équilibrées et des modèles plus avancés (comme XGBoost ou LightGBM).

Globalement, ce travail illustre comment la combinaison de l’analyse de données et du génie télécom peut favoriser une prise de décision intelligente et améliorer la fiabilité du réseau et l’expérience utilisateur.

Schlussfolgerung

Dieses Miniprojekt zeigt, wie datenanalysebasierte maschinelle Lernmethoden effektiv zur Fehlerbehebung und Leistungsoptimierung von Telekommunikationsnetzen eingesetzt werden können.

Durch einen strukturierten Prozess – von der Datensammlung und -bereinigung über die Visualisierung bis hin zur Diagnose und Modelloptimierung – wurden wertvolle Erkenntnisse über Signalstärke, Umwelteinflüsse und Bereiche mit schwacher Konnektivität gewonnen.

Obwohl das Random-Forest-Modell eine mittlere Genauigkeit ($\approx 42\%$) erzielte, identifizierte es wichtige Leistungsmuster und zeigte die Notwendigkeit ausgewogenerer Daten und fortschrittlicherer Modelle (wie XGBoost oder LightGBM).

Insgesamt verdeutlicht diese Studie, wie die Verbindung von Datenanalyse und Telekommunikationstechnik zu einer intelligenten Entscheidungsfindung beiträgt und die Netzzuverlässigkeit und Benutzererfahrung verbessert.